橋脚用ワイヤ吊り下げ式ロボット
制御システム仕様書

版：試作 v1.0（空転試験前）　作成日：2026年10月27日
対象：制御用電装（Raspberry Pi 4B・IBT-2・IMU・PS5コントローラ）および制御プログラム

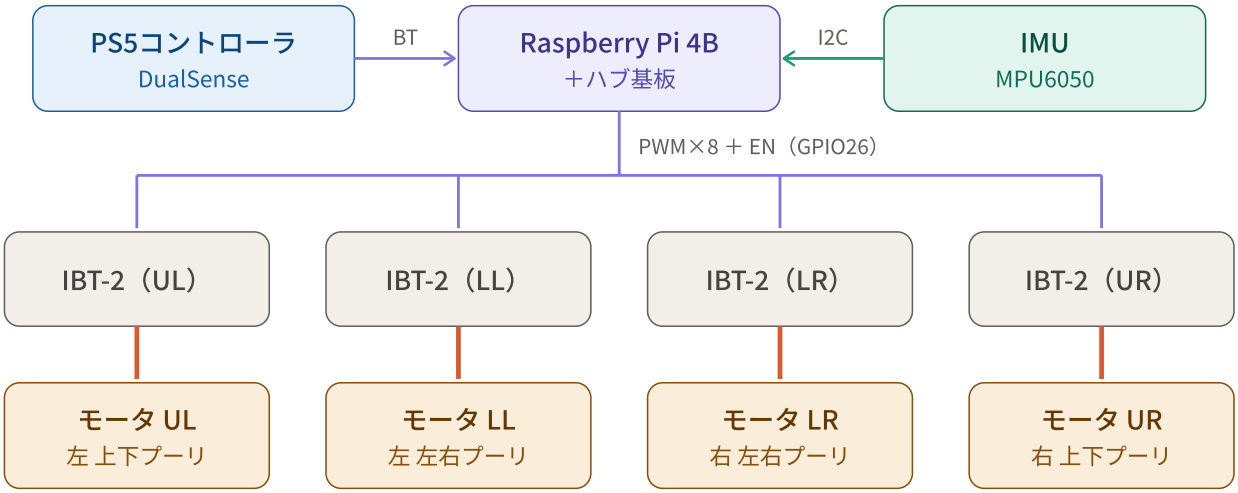
1. 概要
2. システム構成
3. 機体内の配置
4. 電源系
5. 信号系配線
6. ソフトウェア
7. 操作仕様
8. 安全機能
9. セットアップ・試験手順
10. 既知の制約・今後の課題
11. 付録：設定パラメーター一覧

1. 概要

橋脚の塗膜をIH式塗膜剥離装置で加熱・剥離するために、装置を壁面に沿って移動させるワイヤ吊り下げ式ロボットの制御システムである。機体上方のレールに2つのレールランナーを載せ、機体側の4つのドラムでワイヤを巻き取り・巻き出しすることで、上下・左右に移動する。

動作	原理
上下移動	4本のワイヤを同じ方向に巻き取り／巻き出しする
左右移動	上下プーリ（外側2本）は停止したまま、左右プーリ（内側2本）を逆回転させる（一方を巻き取り、他方を巻き出し）
姿勢維持	IMUで機体の傾きを測り、上下プーリ（外側）の回転差で水平に戻す
保持	停止中はモータに電流を流さず、ウォームギアのセルフロックで保持する

2. システム構成



12V 直交軸ウォームギアモータ（60kg·cmクラス）× 4 → ドラム → ワイヤ → レールランナー

図1　システム構成

構成品	型式・仕様	数量・備考
制御用コンピュータ	Raspberry Pi 4B	1。OS：Raspberry Pi OS（64bit, Lite）
モータドライバ	IBT-2（BTS7960）	4
モータ	12V 直交軸ウォームギアモータ（60kg·cmクラス）	4。エンコーダなし
IMU	MPU6050（6軸：加速度＋ジャイロ）	1。I2C、アドレス 0x68
操作端末	PS5 DualSense コントローラ	1。Bluetooth（USB接続も可）
バッテリー	リポ	1
DCDCコンバータ	5V 3A以上（出力 5.1V 程度に調整）	1。ラズパイ用
モータ電源スイッチ	DCブレーカー 15A	1。スイッチ兼過電流保護、物理的な非常停止
ラズパイ電源スイッチ	トグルスイッチ DC 3A以上	1
ハブ基板	ユニバーサル基板	1。ラズパイ上に搭載、3.3V・GND・EN・PWM を分配、IMU を搭載

モータとワイヤの対応

記号	名称	ワイヤ
UL	左 上下プーリ	左 外側ワイヤ
UR	右 上下プーリ	右 外側ワイヤ
LL	左 左右プーリ	左 内側ワイヤ（レールランナーを経由）
LR	右 左右プーリ	右 内側ワイヤ（レールランナーを経由）

3. 機体内の配置

- 中央上部：Raspberry Pi 4B、その上にハブ基板（IMUを搭載）
- 左右両側：IBT-2 を上下2段ずつ（左：UL・LL、右：UR・LR）
- リポ：作業者側の面の下部・左右中央（下側モータの下）
- ブレーカー・スイッチ：リポ付近の手が届く位置

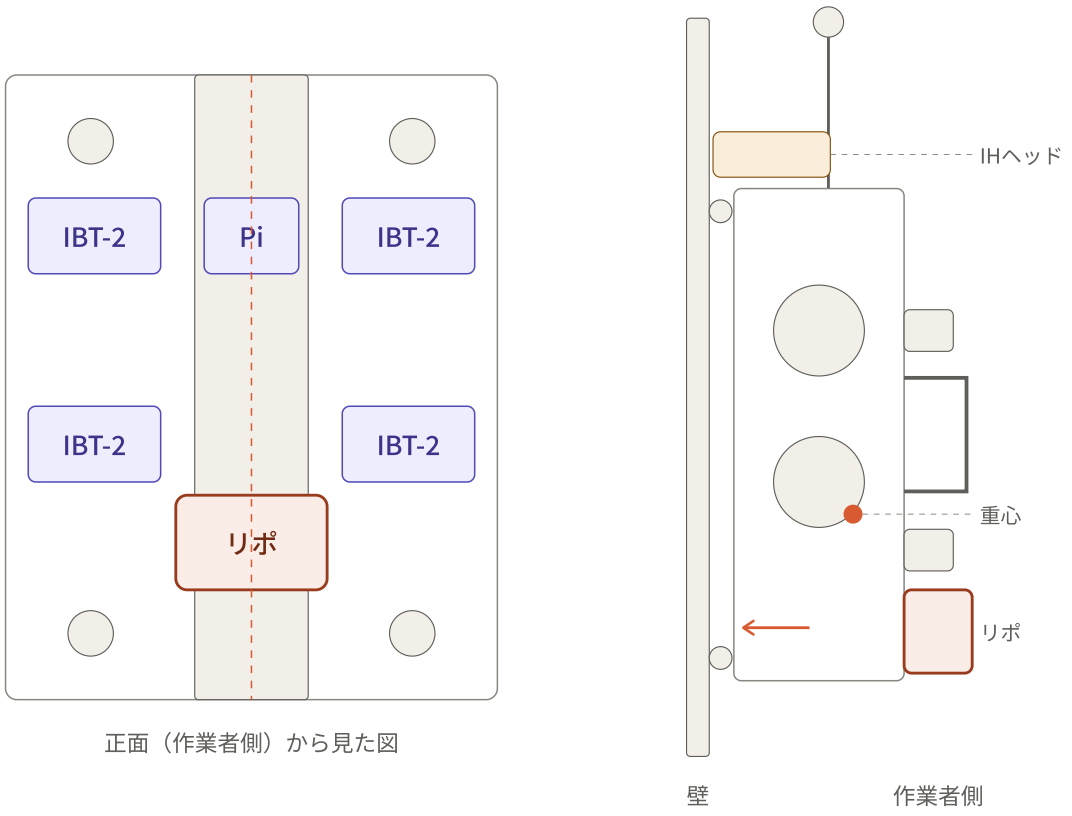


図2　配置（左：正面図、右：側面図）

リポの配置理由

- 左右中央：重心が左右にずれると、停止中も傾いたまま吊られるため
- 下部：重心が吊り点から下に離れるほど姿勢が安定するため
- 作業者側：重心がワイヤより外側に来て、機体下部が壁に押し付けられ、磁石の補助になるため

- IHヘッド（上部・壁側）から離す：リポの外装はアルミラミネートで、IHの磁界で加熱される恐れがあるため
- 配線：上下のIBT-2の中間に近く、モータ電源の幹線が短くなるため

IHヘッドの位置と下部シリンダの役割は、構想図とCADからの推定である。実機と異なる場合は配置を見直す。

4. 電源系

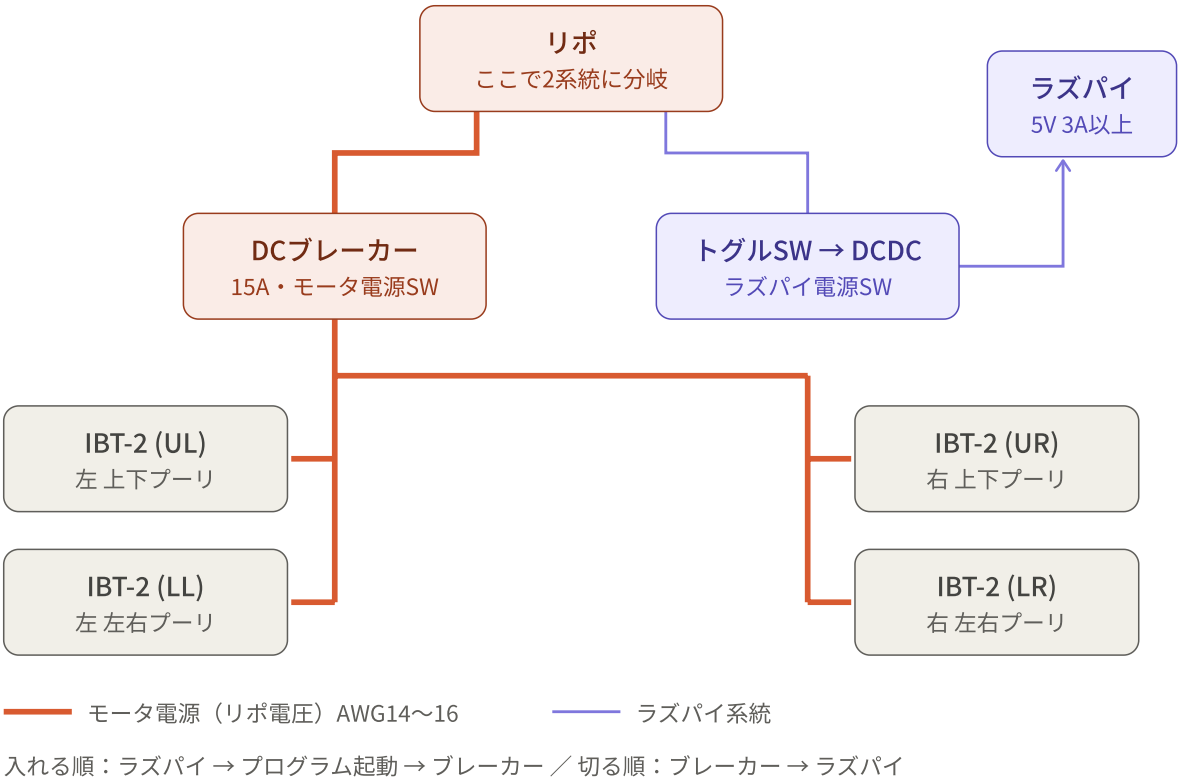


図3　電源系（スイッチ2系統）

部品・配線	目安
モータ側ブレーカー	DC 15A（起動時に落ちる場合は 20A）
幹線（リポ〜ブレーカー〜分岐）	AWG14〜16
分岐〜各IBT-2、IBT-2〜モータ	AWG18
ラズパイ側スイッチ	DC 3A以上
DCDC	5V 3A以上、出力 5.1V 程度

電流の見積もり（12V、1台あたり、概算）

状態	電流
無負荷	0.5A 前後
通常運転（機体を吊って昇降）	1〜2A（4台合計 5〜8A）
ロック時	4〜6A 程度（4台同時ロックで約 20A）

> 電流値は一般的な仕様からの概算である。モータ銘板の定格電流・ロック電流を確認し、可能ならクランプメータで実測してブレーカー定格を確定する。

運用ルール

- スイッチは必ず DC定格のものを使う（AC用はDC大電流で接点が溶けることがある）
- GND はリポ付近の1点にまとめる（スター配線）
- 電源投入順：ラズパイ → プログラム起動 → ブレーカー。遮断順：ブレーカー → ラズパイ
- ブレーカーを物理的な非常停止として使う
- Pi 4B は電圧低下に弱い。`vcgencmd get_throttled` が `0x0` 以外なら電源を見直す
- IH装置の電源系統とロボット側の電源・GNDは接続しない

5. 信号系配線

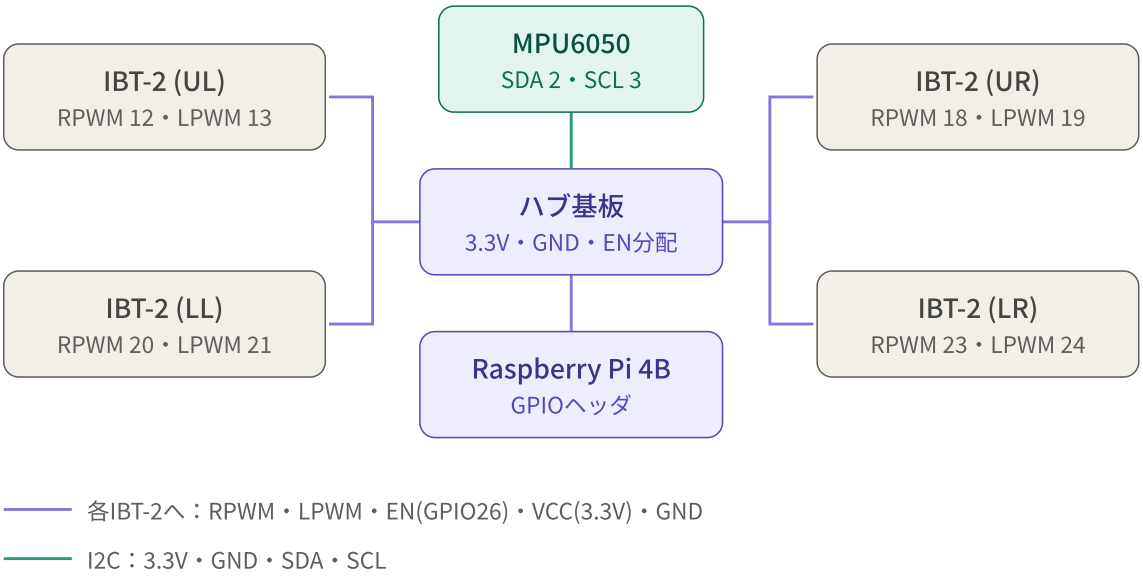


図4　信号系

IBT-2 のピン

ピン	使用	接続先
RPWM	○	ラズパイGPIO（モータごと、下表）
LPWM	○	ラズパイGPIO（モータごと、下表）
R_EN	○	GPIO26（4台共通。IBT-2上で L_EN とつなぐ）
L_EN	○	同上
VCC	○	ラズパイ 3.3V（物理1番 または 17番）
GND	○	ラズパイ GND
R_IS / L_IS	×	未接続（電流検出。将来の過電流停止用）
B+ / B−	○	モータ電源（B− は太い線でリポの − に直接）
M+ / M−	○	モータ

ラズパイのピン割り当て（BCM番号／物理ピン）

信号	RPWM	LPWM
UL（左 上下プーリ）	GPIO12／32番	GPIO13／33番
LL（左 左右プーリ）	GPIO20／38番	GPIO21／40番
UR（右 上下プーリ）	GPIO18／12番	GPIO19／35番
LR（右 左右プーリ）	GPIO23／16番	GPIO24／18番

共通信号	ラズパイのピン
EN（R_EN・L_EN 4台分）	GPIO26／37番
3.3V（IBT-2 VCC・MPU6050）	1番 または 17番
GND	6・9・14・20・25・30・34・39番のいずれか
I2C（MPU6050）	SDA：GPIO2／3番、SCL：GPIO3／5番

- IBT-2 の VCC は 3.3V とする。5V にすると入力バッファ（74HC244）の H 判定が 3.5V 以上となり、ラズパイの 3.3V 出力では動作が不安定になる
- IBT-2 の基板内で信号 GND と B− がつながっているため、ラズパイ GND とリポ − の間に別線は不要。B− を細くするとモータ電流が信号 GND に流れ、ラズパイのリセットの原因になる
- モータの回転方向が逆の場合は M+／M− を入れ替えるか、`config.py` の `invert` を変更する

EN の配線方式

方式	配線	config.py
GPIO26で制御（推奨）	R_EN＋L_EN → GPIO26	`ENABLE_PIN = 26`
VCCに直結	IBT-2上で VCC・R_EN・L_EN をつなぐ	`ENABLE_PIN = None`

どちらの方式でも通常の停止は同じように効く。GPIO26方式は、待機中にPWM線へノイズが乗っても回らないこと、将来ウォッチドッグ回路をEN線に入れられることが利点である。プログラムの強制終了やラズパイのフリーズ時は、どちらの方式でも止まらない可能性があるため、最終的な非常停止はブレーカーで行う。

IMU の取り付け

- ハブ基板上に固定し、ラズパイ本体もフレームにしっかり固定する（ぐらつくと角度が暴れる）
- CPUの熱、DCDC、モータ、IBT-2 から少し離す
- 基板の向きは機体の上下・左右にそろえる（ずれは `IMU_AXIS_*` で補正可能）

6. ソフトウェア

ファイル構成

ファイル	内容
`config.py`	ピン番号・回転方向・速度・ゲイン・ボタン割り当て・ライトバー色。基本的にここだけ編集する
`main.py`	操縦プログラム本体（50Hz ループ）
`mixer.py`	スティック入力 → 4ドラムの指令値への変換
`controller.py`	傾き補正の PI 制御、加速制限
`motors.py`	モータドライバ（IBT-2。他ドライバへも切替可）、EN 制御
`imu.py`	MPU6050 の読み取り、相補フィルタによる傾き推定
`gamepad.py`	コントローラ入力（evdev）、自動再接続、ライトバー、振動
`tools/test_motors.py`	モータを1台ずつ回して配線・巻き取り方向を確認
`tools/imu_check.py`	IMU の取り付け向きを確認
`tools/gamepad_check.py`	コントローラのボタン名を確認
`tests/test_logic.py`	ハードなしで動くロジックのテスト

起動方法

```
python3 main.py                 実機
python3 main.py --sim           モータ・IMU をダミーにして動作確認（コントローラは実物）
python3 main.py --log run.csv   制御ログを CSV に保存（調整用）
```

符号の約束

- モータ指令：+1.0 ＝ 巻き取り、−1.0 ＝ 巻き出し
- 傾き θ：正面（作業者側）から見て右側が下がる向きを正

指令の合成（走行モード）

上下指令 vy、左右指令 vx、傾き補正量 corr から、各モータの指令を次のように計算する。いずれかが ±1 を超える場合は、比率を保ったまま全体を縮める。

UL（左外）	UR（右外）	LL（左内）	LR（右内）
vy − corr	vy ＋ corr	vy ＋ vx	vy − vx

動作	UL	UR	LL	LR
上昇	巻取	巻取	巻取	巻取
右移動	停止	停止	巻取	巻出
傾き補正（右が下がった）	少し巻出	少し巻取	—	—

左右移動の向き（`LATERAL_SIGN`）は資料からの推定であり、実機で逆なら −1 にする。

傾き補正

- 相補フィルタ（ジャイロの重み 0.98）で傾きを推定し、PI 制御で補正量を計算する
- 補正はスティックを倒している間だけ効く。停止中はモータを一切動かさない
- 傾きが 12° を超えたら移動を止め、補正だけを行う
- △ボタンで ON/OFF、CREATE ボタンで現在の姿勢を水平として記録できる

7. 操作仕様（PS5コントローラ）

スティックを ±0.2 より大きく倒している間だけ動き、離せば止まる。押しっぱなしにするボタンはない。モードは走行モードとワイヤ調整モードの2つで、PSボタンで切り替える。

共通

ボタン	動作
PSボタン	走行モード ⇔ ワイヤ調整モード（スティックを離した状態で。切り替え時に振動）
×	非常停止（ラッチ）
OPTIONS	非常停止の解除（スティックを離した状態で）

ライトバーの色

色	状態
青	走行モード
オレンジ	ワイヤ調整モード
赤	非常停止中
点滅	スティックが中央に戻るのを待っている（離せば点灯に戻る）

走行モード（青）

入力	動作
左スティック 上下	上昇／下降（4本同時）
右スティック 左右	左右移動
R1	押している間は高速
△	傾き補正 ON/OFF
CREATE	現在の姿勢を水平として記録

上下と左右は同時に操作でき、斜めに移動する（`AXIS_LOCK = True` で大きく倒した方だけを有効にできる）。

ワイヤ調整モード（オレンジ）

肩ボタンの位置が機体のドラムの位置に対応する。1回押すとそのモータが選択され（短く振動）、別のボタンを押すとそちらに切り替わる。

ボタン	位置	選択されるモータ
L1	左上	LL（左 左右プーリ）
R1	右上	LR（右 左右プーリ）
L2	左下	UL（左 上下プーリ）
R2	右下	UR（右 上下プーリ）

入力	動作
左スティック 上	選択中のモータを巻き取り（倒し具合で速度が変わる）
左スティック 下	選択中のモータを巻き出し

- モードに入った直後は何も選ばれておらず、スティックを倒しても動かない
- モータの切り替えはスティックを離した状態で行う
- 傾き補正は停止する
- 上段が左右プーリ、下段が上下プーリという配置は推定。実機と異なる場合は `WIRE_BUTTONS` で入れ替える

8. 安全機能

状況	動作
×ボタン	非常停止（ライトバー赤）。スティックを離して OPTIONS で解除
スティックを離す（±0.2以内）	全モータ即停止、EN を L
コントローラ切断（OSが切断を検出したとき）	全モータ即停止
Ctrl+C／プログラムの通常終了	全モータ停止、EN を L、ピン解放
起動直後・モード切替・非常停止解除・再接続の後	スティックが一度中央に戻るまで動かない（急発進防止）
ワイヤ調整中にスティックを倒したまま別モータを選択	受け付けない
傾き 12° 超（走行モード）	移動を止め、補正のみ
動き始め	加速を制限（停止は即時）
ラズパイ起動中	GPIO26 がプルダウンのため、モータは回らない
上記で止まらない場合	ブレーカーを切る（物理的な非常停止）

Bluetooth の切断は、通信が途切れてから OS が切断と判定するまで数秒かかることがあり、その間は最後のスティック操作が継続する。ワイヤを張って吊るす試験の前に、通信途絶を 0.2 秒程度で検出して停止する機能を追加することを推奨する（第10章）。

9. セットアップ・試験手順

OS とライブラリ

```
Raspberry Pi Imager で Raspberry Pi OS（64bit, Lite）を書き込む
  （詳細設定でユーザー名・Wi-Fi・SSH を設定）
sudo raspi-config        # Interface Options → I2C を有効化
sudo apt install python3-gpiozero python3-lgpio python3-evdev python3-smbus2 i2c-tools
i2cdetect -y 1           # 0x68 が見えれば IMU 接続 OK
sudo usermod -aG input,gpio,i2c $USER   # 再ログイン後 sudo なしで動く
```

最近の Raspberry Pi OS では `pip install` でのシステムへのインストールはエラーになるため、apt を使う。

PS5コントローラ

```
bluetoothctl
  scan on
  （コントローラの CREATE + PS を長押し、ライトバーが速く点滅）
  pair XX:XX:XX:XX:XX:XX
  trust XX:XX:XX:XX:XX:XX
  connect XX:XX:XX:XX:XX:XX
```

ライトバーの色を変えるには、LED の書き込み権限を与える udev ルールを追加する（手順は README.md を参照）。追加しなくても操縦はできる。

空転試験（ワイヤなし）

1. コントローラ：`tools/gamepad_check.py` でボタン名を確認
2. モータ：ドラムにテープで印を付け、`tools/test_motors.py` で「+」が巻き取り方向か確認。逆なら `invert` を反転
3. 走行モード：左スティック上で4台とも巻き取り、右スティック右で LL 巻取・LR 巻出・UL/UR 停止
4. ワイヤ調整モード：L1・R1・L2・R2 で狙ったドラムだけが回る
5. 安全：×・スティックを離す・コントローラ電源オフ・ブレーカーで止まる
6. IMU：`tools/imu_check.py` で手で傾けて符号を合わせる
7. 傾き補正の向き：左スティックを少し上に倒したまま右側を手で下げ、UR が UL より速く回る
8. ライトバー：モードに応じて色が変わる

空転は回転が速いので、最初は低速（R1 なし）で行い、配線や指の巻き込みに注意する。`--log` でログを残す。

吊り下げ試験（ワイヤあり）で確認すること

- ワイヤ調整モードで4本の張りを揃える
- 低い位置で、水平に戻るか（傾き補正ゲインの調整）
- 左右移動の向き（`LATERAL_SIGN`）
- 負荷時の電流、低速での回り出し（`MIN_DUTY`）
- IH 作動中に、通信の途切れ・ラズパイの停止・IMU の値の飛びがないか

傾き補正の調整

`TILT_KI = 0` で `TILT_KP` を上げていき、揺れ始めたら半分に戻す。その後 `TILT_KI` を少し足す。

10. 既知の制約・今後の課題

項目	内容・対応案
ワイヤ長の同期	エンコーダがないため4本の長さを同期できない。ずれはワイヤ調整モードで合わせる。将来はエンコーダ付きモータで速度制御を追加
高さのずれ	左右移動中の高さのずれは補正できない（IMU で分かるのは傾きのみ）
通信途絶の検出	コントローラが常時送るモーションセンサ値を生存確認に使い、0.2 秒途切れたら停止する機能を追加する（推奨・未実装）
IH のノイズ	基板を載せる板をアルミ板（1.5〜2mm、GND に1点接続、基板はスペーサーで浮かせる）にする。信号線のより線化・フェライトコア。IH 装置本体をアルミで覆うのは加熱・性能低下の恐れがあるため不可。メーカーに周囲金属の距離と漏れ磁界対策を確認する
過電流停止	IBT-2 の R_IS／L_IS と ADC（ADS1115 等）で電流を測り、巻き切り時に停止する機能を追加できる
ウォッチドッグ	EN 線にウォッチドッグ回路を入れ、ソフトが固まっても EN を切れるようにする
電流値	概算値。銘板確認と実測でブレーカー定格を確定する
推定事項の確認	ドラム配置（上段＝左右プーリ）、`LATERAL_SIGN`、IH ヘッドの位置
バッテリー監視	ラズパイからリポ電圧を測れないため、リポアラームを付ける
自動起動	電源投入でプログラムが起動するよう systemd に登録する（必要に応じて）

11. 付録：設定パラメーター一覧（config.py）

パラメータ	既定値	内容
`MOTOR_DRIVER_MODE`	"ibt2"	モータドライバの種類
`PWM_FREQUENCY`	1000	PWM 周波数 [Hz]
`ENABLE_PIN`	26	EN のピン。VCC 直結なら None
`MOTORS`	—	各モータの RPWM／LPWM ピンと回転方向反転
`LATERAL_SIGN`	+1	左右移動の向き
`MAX_DUTY`	0.8	最大デューティ
`LOW_SPEED_SCALE`	0.4	通常時の速度倍率（R1 で 1.0）
`MIN_DUTY`	0.12	回り始めの底上げ
`SLEW_RATE`	2.0	加速の上限 [デューティ/秒]
`WIRE_ADJUST_SCALE`	0.3	ワイヤ調整モードの速度
`WIRE_BUTTONS`	—	肩ボタンとモータの対応
`TILT_CORRECTION_DEFAULT_ON`	True	起動時の傾き補正
`TILT_KP`	0.04	比例ゲイン [デューティ/deg]
`TILT_KI`	0.01	積分ゲイン [デューティ/(deg·s)]
`TILT_I_LIMIT`	20.0	積分項の上限 [deg·s]
`TILT_MAX_CORRECTION`	0.3	補正量の上限
`TILT_DEADBAND_DEG`	0.5	無視する傾き [deg]
`TILT_LIMIT_DEG`	12.0	移動を止める傾き [deg]
`COMPLEMENTARY_ALPHA`	0.98	相補フィルタのジャイロの重み
`IMU_AXIS_RIGHT／UP／NORMAL`	+x／+y／+z	IMU の取り付け向き
`STICK_DEADZONE`	0.2	スティックのデッドゾーン
`AXIS_LOCK`	False	True で上下・左右の同時操作を禁止
`COLOR_DRIVE／WIRE／ESTOP`	青／橙／赤	ライトバーの色
`RUMBLE_MS／SELECT_RUMBLE_MS`	200／80	振動の長さ [ms]
`LOOP_HZ`	50	制御周期 [Hz]